



UNIVERSIDAD TECNOLÓGICA SANTA CATARINA

MATERIA: Aplicaciones web orientada a servicios

ACTIVIDAD: Practica de laboratorio de la teoría al endpoint

ALUMNO: Daniel Salazar Pascuala

MATRÍCULA: 25894

GRUPO: DSM-05ª

Etapa 1

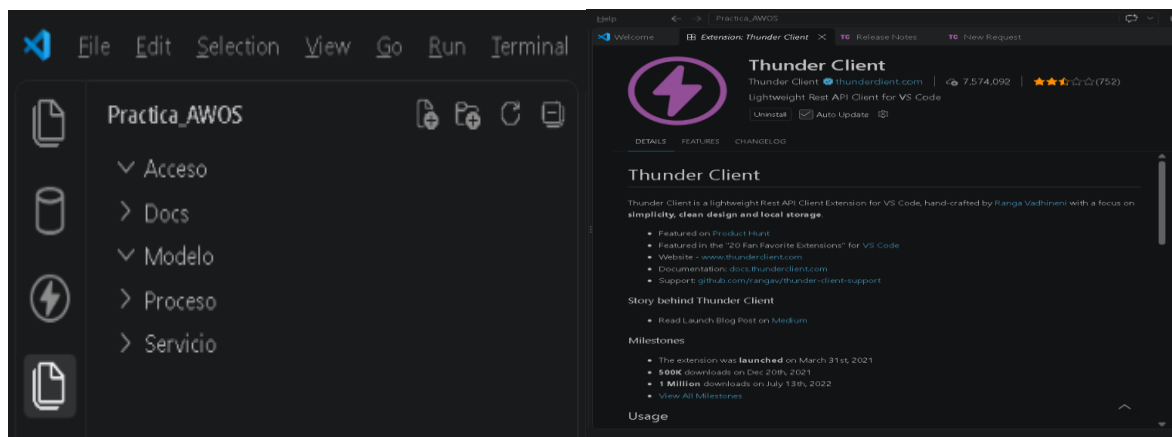
Se organizó el proyecto en 4 capas desacopladas para cumplir con los lineamientos de arquitectura del curso:

Acceso: Aloja la interfaz pública (index.html). Captura las búsquedas del usuario y despliega la información final en pantalla sin conocer cómo ni de dónde se obtienen los datos.

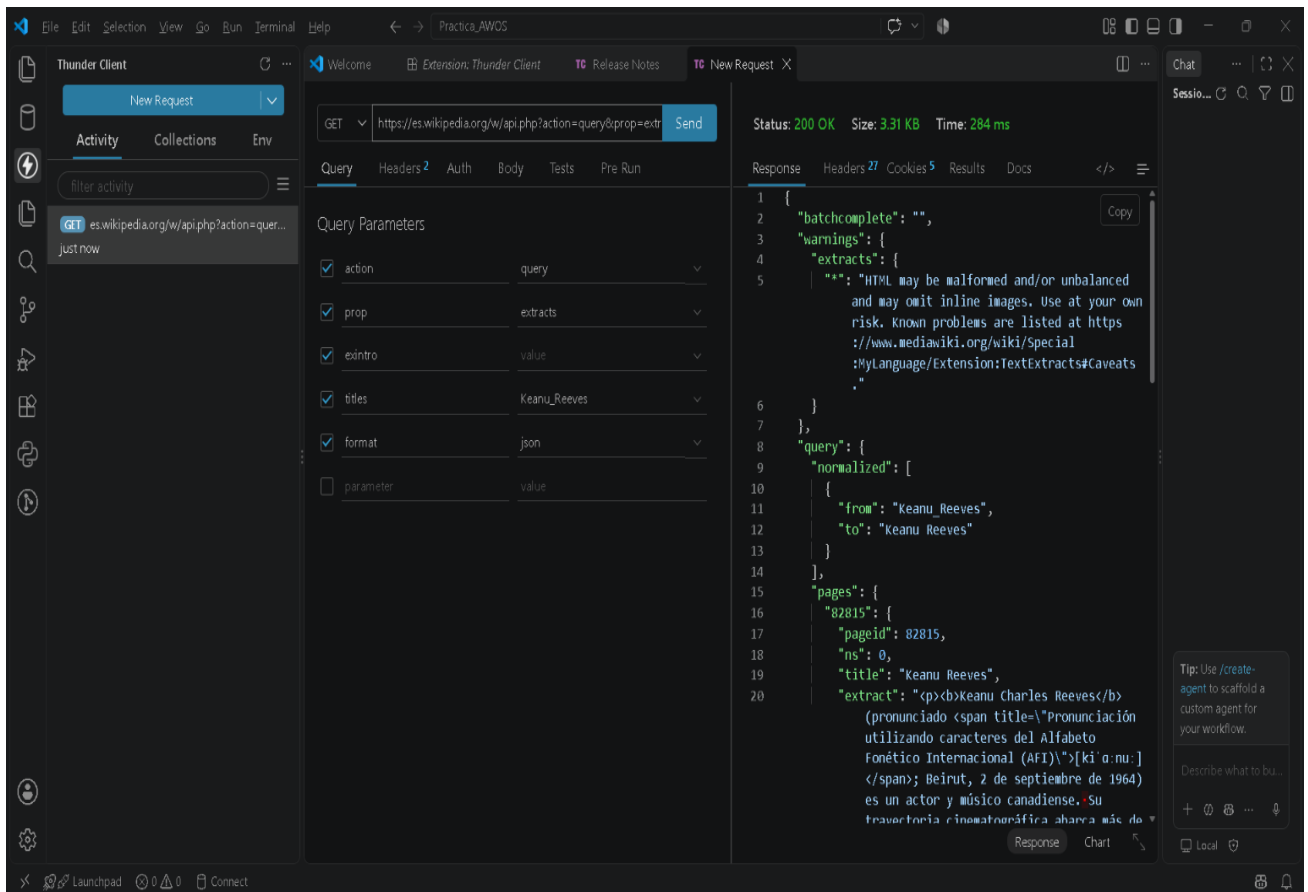
Procesos: Contiene el archivo orquestador (orquestador.js). Su función es coordinar la lógica de negocio y encadenar la salida del primer servicio como entrada del segundo.

Servicios: Aloja los módulos clientes (omdbCliente.js y wikipediaCliente.js). Cada módulo aísla la comunicación con un proveedor externo específico.

Modelo: Define las estructuras de datos estandarizadas para aislar a la aplicación de los nombres originales de las propiedades de los proveedores.



Se instaló la extensión **Thunder Client** en Visual Studio Code para disponer de un cliente HTTP integrado. Esto permite invocar los endpoints REST, verificar los parámetros requeridos y analizar las respuestas JSON antes de iniciar la etapa de codificación.



https://es.wikipedia.org/w/api.php?action=query&prop=extracts&exintro&titles=Keanu_Reeves&format=json

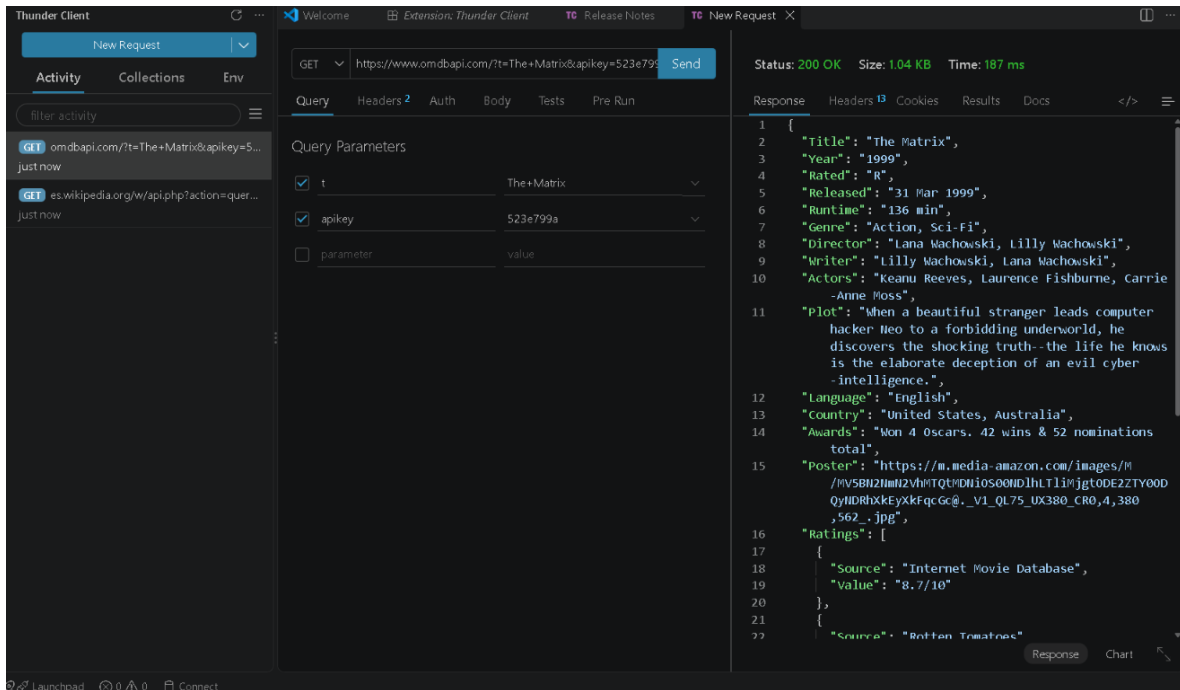
1. Ficha Técnica: API Wikipedia (Biografías)

- **Proveedor:** Wikimedia Foundation
- **Endpoint:** https://es.wikipedia.org/w/api.php](https://es.wikipedia.org/w/api.php)
- **Estándar / Formato:** REST / JSON
- **Parámetros:** action=query, prop=extracts, titles=Keanu_Reeves, format=json
- **Autenticación / Límites:** Sin autenticación / 200 peticiones/seg
- **Fuente oficial:** https://www.mediawiki.org/wiki/API:Main_page/es
https://www.mediawiki.org/wiki/API:Main_page/es

2. Ficha Técnica: API OMDb (Películas)

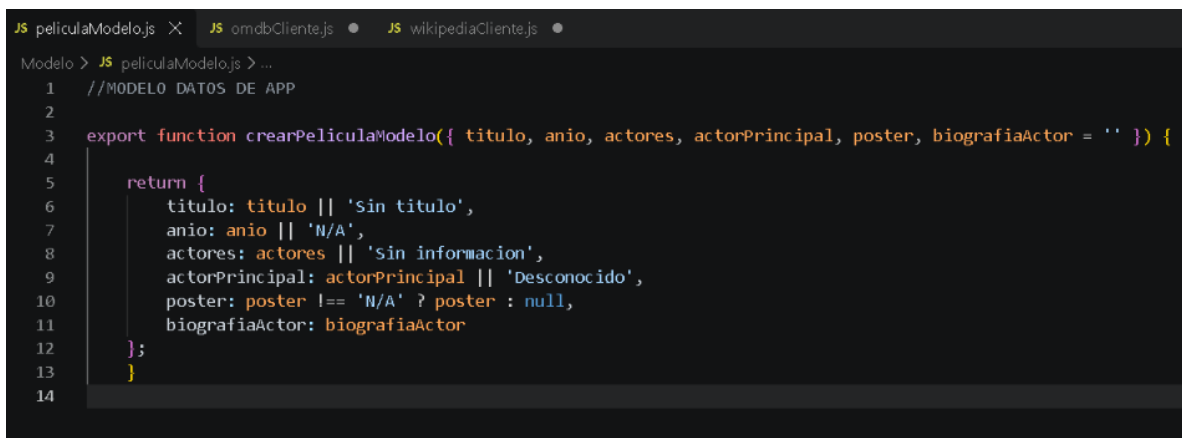
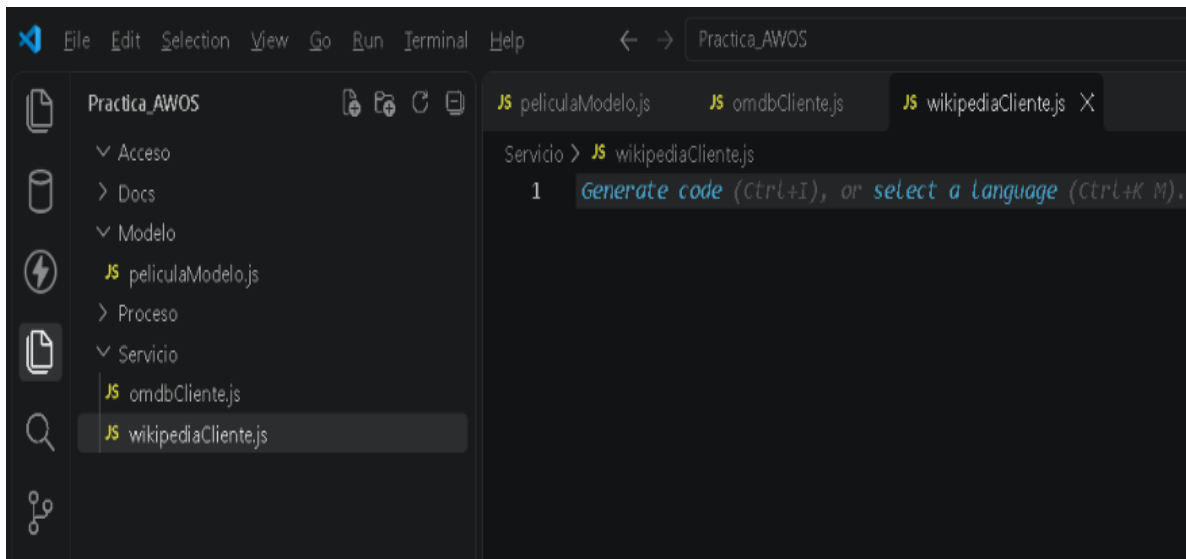
- **Proveedor:** Open Movie Database
- **Endpoint:** https://www.omdbapi.com/](https://www.omdbapi.com/)

- **Estándar / Formato:** REST / JSON
- **Parámetros:** t=The+Matrix, apikey=523e799a
- **Autenticación / Límites:** API Key requerida / 1,000 peticiones por día
- **Fuente oficial:** <https://www.omdbapi.com/>](<https://www.omdbapi.com/>



Se comprobó que el campo Actors retornado por OMDB entrega el nombre del actor (Keanu Reeves), el cual sirve directamente como valor del parámetro titles para consultar la biografía en Wikipedia."

ETAPA 2



```
JS pelculaModelo.js JS omdbCliente.js JS wikipediaCliente.js
Servicio > JS omdbCliente.js > ...
1
2 import { crearPeliculaModelo } from '../modelos/peliculaModelo.js';
3 const API_KEY = '523e799a';
4
5 export async function buscarPelicula(titulo) {
6   try {
7
8     const url = `https://www.omdbapi.com/?t=${encodeURIComponent(titulo)}&apikey=${API_KEY}`;
9     const respuesta = await fetch(url);
10
11     if (!respuesta.ok) {
12       throw new Error(`Servidor OMDb no disponible (Código: ${respuesta.status})`);
13     }
14
15     const datos = await respuesta.json();
16
17     if (datos.Response === 'False') {
18       throw new Error("No se encontro esta pelicula con este titulo");
19     }
20
21     const actorPrincipal = datos.Actors ? datos.Actors.split(',')[0].trim() : 'Desconocido';
22
23     return crearPeliculaModelo({
24       titulo: datos.Title,
25       anio: datos.Year,
26       actores: datos.Actors,
27       actorPrincipal: actorPrincipal,
28       poster: datos.Poster
29     });
30
31   } catch (error) {
32     throw new Error(`[omdbCliente] ${error.message}`);
33   }
34 }
```

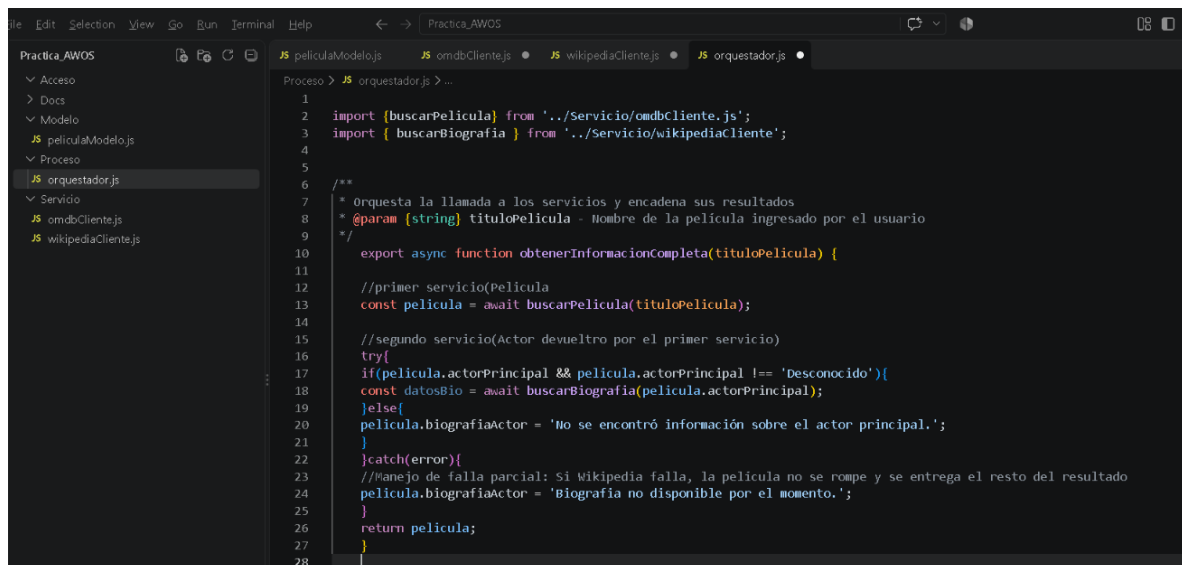
```
JS pelculaModelo.js JS omdbCliente.js JS wikipediaCliente.js
Servicio > JS wikipediaCliente.js > buscarBiografiaActor
1
2 export async function buscarBiografiaActor(actor) {
3   try {
4
5     const url = `https://es.wikipedia.org/w/api.php?action=query&prop=extracts&exintro&titles=${encodeURIComponent(nombreActor)}&format=json`;
6     const respuesta = await fetch(url);
7
8     if (!respuesta.ok) {
9       throw new Error(`Servidor de Wikipedia no disponible (código: ${respuesta.status})`);
10    }
11
12    const datos = await respuesta.json();
13    const paginas = datos.query.pages;
14    const pageId = Object.keys(paginas)[0];
15
16    // Si pageId es "-1", Wikipedia no encontró el artículo
17    if (pageId === "-1") {
18      throw new Error(`No se encontró la biografía para: ${nombreActor}`);
19    }
20
21    // DESACOMPLIMIENTO: Entregamos únicamente lo que la app necesita
22    return {
23      biografia: paginas[pageId].extract
24    };
25
26  } catch (error) {
27    throw new Error(`[wikipediaCliente] ${error.message}`);
28  }
29 }
```

Evidencia de la Capa de Servicios y Modelo: Se presentan los archivos `peliculaModelo.js`, `omdbCliente.js` y `wikipediaCliente.js`. Se aplica el principio de desacoplamiento al traducir las respuestas de los proveedores externos a un modelo de datos propio y encapsular el control de errores HTTP dentro de sus respectivos módulos.

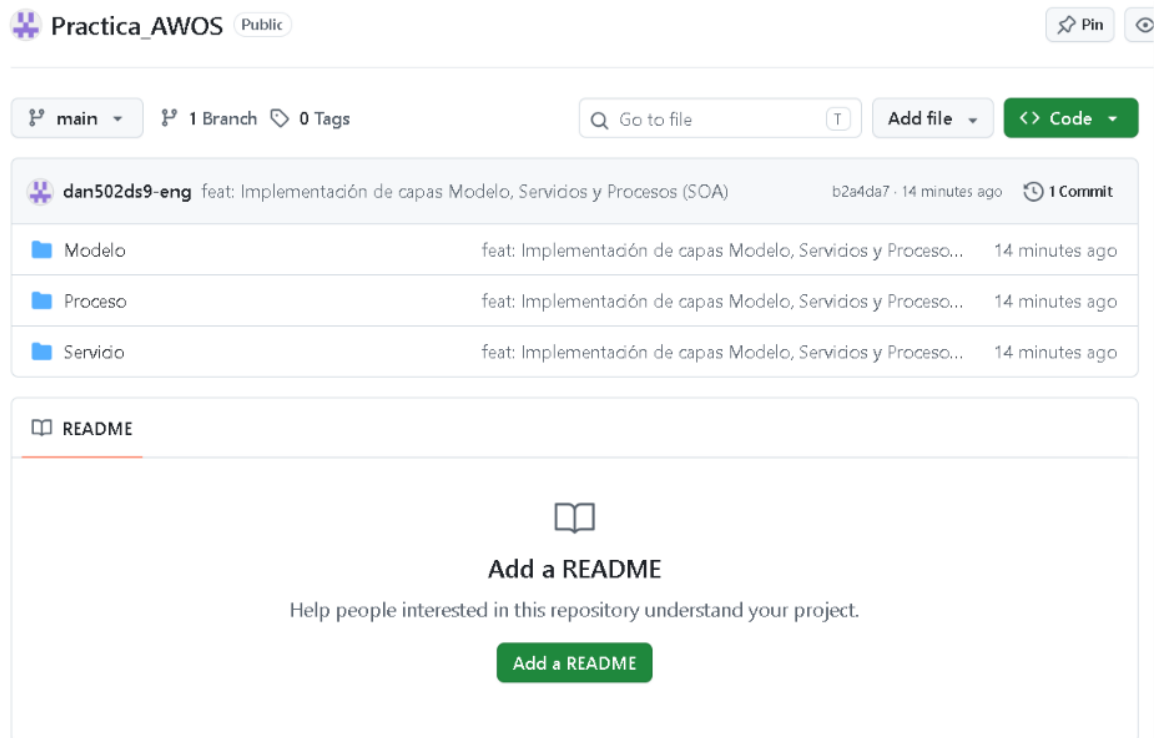
ETAPA 3

Capa de Procesos:

En la Capa de Procesos `orquestador.js` se implementa la lógica de composición sin realizar peticiones HTTP directas. Se encadena la salida del cliente **“OMDb actorPrincipal”** como parámetro de entrada para el cliente Wikipedia, resolviendo las fallas parciales para entregar un objeto unificado a la interfaz de usuario.



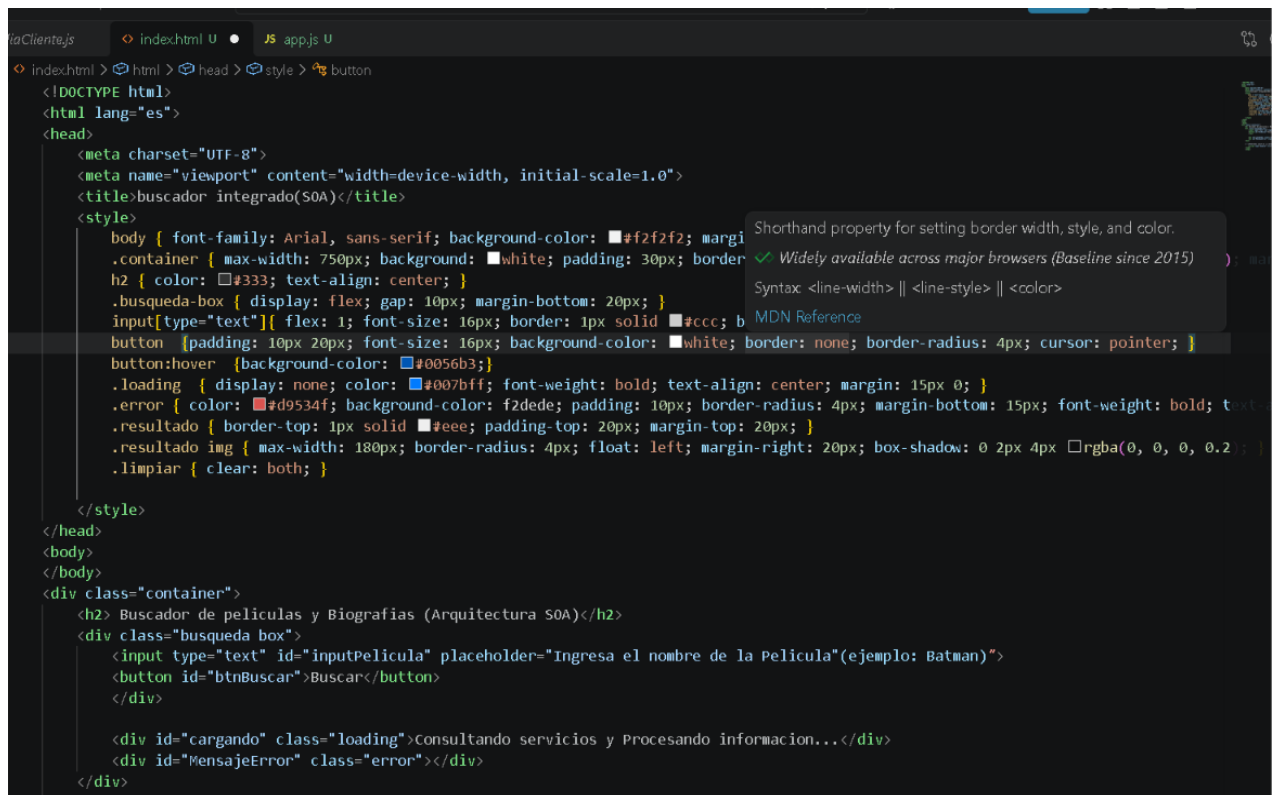
```
1
2 import {buscarPelicula} from '../Servicio/omdbCliente.js';
3 import { buscarBiografia } from '../Servicio/wikipediaCliente.js';
4
5
6 /**
7  * Orquesta la llamada a los servicios y encadena sus resultados
8  * @param {string} tituloPelicula - Nombre de la película ingresado por el usuario
9  */
10 export async function obtenerInformacionCompleta(tituloPelicula) {
11
12     //primer servicio(Película
13     const pelicula = await buscarPelicula(tituloPelicula);
14
15     //segundo servicio(Actor devuelto por el primer servicio)
16     try{
17         if(pelicula.actorPrincipal && pelicula.actorPrincipal !== 'Desconocido'){
18             const datosBio = await buscarBiografia(pelicula.actorPrincipal);
19         }else{
20             pelicula.biografiaActor = 'No se encontró información sobre el actor principal.';
21         }
22     }catch(error){
23         //Manejo de falla parcial: Si Wikipedia falla, la película no se rompe y se entrega el resto del resultado
24         pelicula.biografiaActor = 'Biografía no disponible por el momento.';
25     }
26     return pelicula;
27 }
28
```

The image shows a Visual Studio Code terminal window with a dark theme. The left sidebar displays a file explorer with a tree view containing folders like 'Acceso', 'Docs', 'Modelo', 'Proceso', 'Servicio', and files like 'orquestador.js', 'omdbCliente.js', and 'wikipediaCliente.js'. The terminal window is open to the 'Terminal' tab and shows the following commands and output:

```
PS C:\Users\DINOBABY\OneDrive\Escritorio\Practica_AWOS> git add .
>> git commit -m "feat: Implementación de capas Modelo, Servicios y Procesos (SOA)"
PS C:\Users\DINOBABY\OneDrive\Escritorio\Practica_AWOS> git remote add origin https://github.com/dan502ds9-eng/Practica_AWOS.git
>> git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 12 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (9/9), 2.21 KiB | 226.00 KiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/dan502ds9-eng/Practica_AWOS.git
 * [new branch]    main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\DINOBABY\OneDrive\Escritorio\Practica_AWOS>
PS C:\Users\DINOBABY\OneDrive\Escritorio\Practica_AWOS> git remote add origin https://github.com/dan502ds9-eng/Practica_AWOS.git
>> git push -u origin main
error: remote origin already exists.
branch 'main' set up to track 'origin/main'.
Everything up-to-date
PS C:\Users\DINOBABY\OneDrive\Escritorio\Practica_AWOS>
```

Capa de Procesos y Estructura del Repositorio: En la imagen se observa el repositorio remoto en GitHub con las carpetas organizadas arquitectónicamente **Modelo, Proceso, Servicio** junto con la terminal de Visual Studio Code que muestra la ejecución exitosa del control de versiones **git add, git commit y git push**, validando la correcta integración del orquestador y la trazabilidad del desarrollo bajo el modelo SOA.

ETAPA 4



```
index.html > html > head > style > button
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>buscador integrado(SOA)</title>
  <style>
    body { font-family: Arial, sans-serif; background-color: #f2f2f2; margin: 0; padding: 10px; }
    .container { max-width: 750px; background: white; padding: 30px; border: 1px solid #ccc; margin: 0 auto; }
    h2 { color: #333; text-align: center; }
    .busqueda-box { display: flex; gap: 10px; margin-bottom: 20px; }
    input[type="text"] { flex: 1; font-size: 16px; border: 1px solid #ccc; padding: 10px 20px; }
    button { padding: 10px 20px; font-size: 16px; background-color: white; border: none; border-radius: 4px; cursor: pointer; }
    button:hover { background-color: #0056b3; color: white; }
    .loading { display: none; color: #007bff; font-weight: bold; text-align: center; margin: 15px 0; }
    .error { color: #d9534f; background-color: #f2dede; padding: 10px; border-radius: 4px; margin-bottom: 15px; font-weight: bold; }
    .resultado { border-top: 1px solid #eee; padding-top: 20px; margin-top: 20px; }
    .resultado img { max-width: 180px; border-radius: 4px; float: left; margin-right: 20px; box-shadow: 0 2px 4px rgba(0, 0, 0, 0.2); }
    .limpiar { clear: both; }
  </style>
</head>
<body>
</body>
<div class="container">
  <h2> Buscador de películas y Biografías (Arquitectura SOA)</h2>
  <div class="busqueda box">
    <input type="text" id="inputPelícula" placeholder="Ingresa el nombre de la Película"(ejemplo: Batman)">
    <button id="btnBuscar">Buscar</button>
  </div>

  <div id="cargando" class="loading">Consultando servicios y Procesando información...</div>
  <div id="MensajeError" class="error"></div>
</div>
```

Shorthand property for setting border width, style, and color.
✓ Widely available across major browsers (Baseline since 2015)
Syntax: <line-width> || <line-style> || <color>
[MDN Reference](#)

Fragmento de código correspondiente a la interfaz de usuario (**index.html**) dentro de la carpeta /acceso. Esta vista define los elementos HTML de presentación, el campo de entrada (input) para capturar el título ingresado por el usuario, y los contenedores estilizados destinados a mostrar el estado de carga, los avisos de error y el resultado integrado, cumpliendo con la regla de que la capa visual no contiene lógica de negocio.

```

Bun Terminal Help < -> Practica_AWOS Update
$ wikipediaCliente.js index.html U JS app.js U
Acceso > JS app.js > ...
1 //acceso app.js
2 import {obtenerInformacionCompleta} from "../Proceso/orquestador.js";
3
4 const inputPelícula = document.getElementById("inputPelícula");
5 const btnBuscar = document.getElementById("btnBuscar");
6 const cargando = document.getElementById("cargando");
7 const mensajeError = document.getElementById("MensajeError");
8 const resultadoContenedor = document.getElementById("resultadoContenedor");
9
10
11 btnBuscar.addEventListener("click", async () => {
12     const título = inputPelícula.value.trim();
13
14     //Limpiamos los errores y los resultados previos
15
16     mensajeError.style.display = "none";
17     mensajeErrores.textContent = "";
18     resultadoContenedor.innerHTML = "";
19
20     if(título){
21         mensajeError.textContent = "Favor de ingresar un título de película.";
22         mensajeError.style.display = "block";
23         return;
24     }
25     // activamos el indicador de carga
26     cargando.style.display = "block";
27
28     try {
29         //invocamos al orquestador para obtener la información completa de la película
30         const resultado = await obtenerInformacionCompleta(título);
31
32         //pintamos el resultado en el contenedor de resultados
33         resultadoContenedor.innerHTML = `
34             <div>
35                 ${resultado.poster ? `` : ""}
36                 <h3>${resultado.título} (${resultado.año})</h3>

```

Fragmento de código del controlador de la vista (**app.js**). Este módulo se encarga de escuchar la interacción del usuario (el evento clic del botón de búsqueda), activar de manera dinámica el estado de carga, invocar a la Capa de Procesos (**orquestador.js**) y renderizar el objeto de datos unificado en la interfaz, capturando cualquier excepción para mostrar mensajes claros y comprensibles al usuario final.

```

Problems Output Debug Console Terminal Ports
PS C:\Users\DINOBABY\OneDrive\Escritorio\Practica_AWOS> git add .
>> git commit -m "feat: Etapa 4 - Implementacion de la capa de acceso e interfaz de usuario"
PS C:\Users\DINOBABY\OneDrive\Escritorio\Practica_AWOS>

```

```

Problems Output Debug Console Terminal Ports
PS C:\Users\DINOBABY\OneDrive\Escritorio\Practica_AWOS> git add .
>> git commit -m "feat: Etapa 4 - Implementacion de la capa de acceso e interfaz de usuario"
PS C:\Users\DINOBABY\OneDrive\Escritorio\Practica_AWOS> git push origin main
Everything up-to-date
PS C:\Users\DINOBABY\OneDrive\Escritorio\Practica_AWOS>

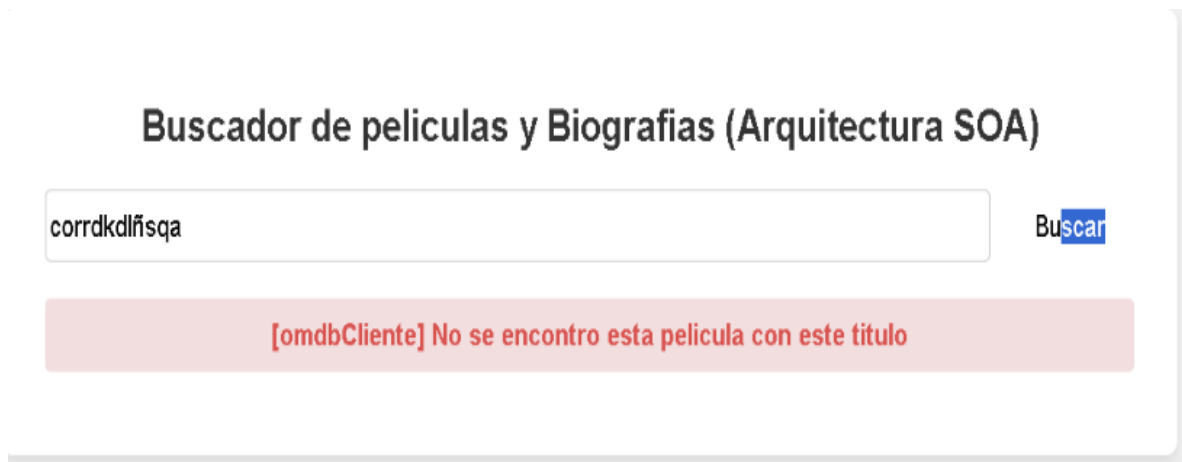
```

Evidencia del control de versiones en la terminal correspondiente al cierre de la Etapa 4. Se muestra la ejecución correcta de los comandos git add, git commit y git push origin

main, reflejando la subida y sincronización exitosa de los archivos de la Capa de Acceso hacia el repositorio remoto en GitHub, lo que garantiza la trazabilidad del proyecto.

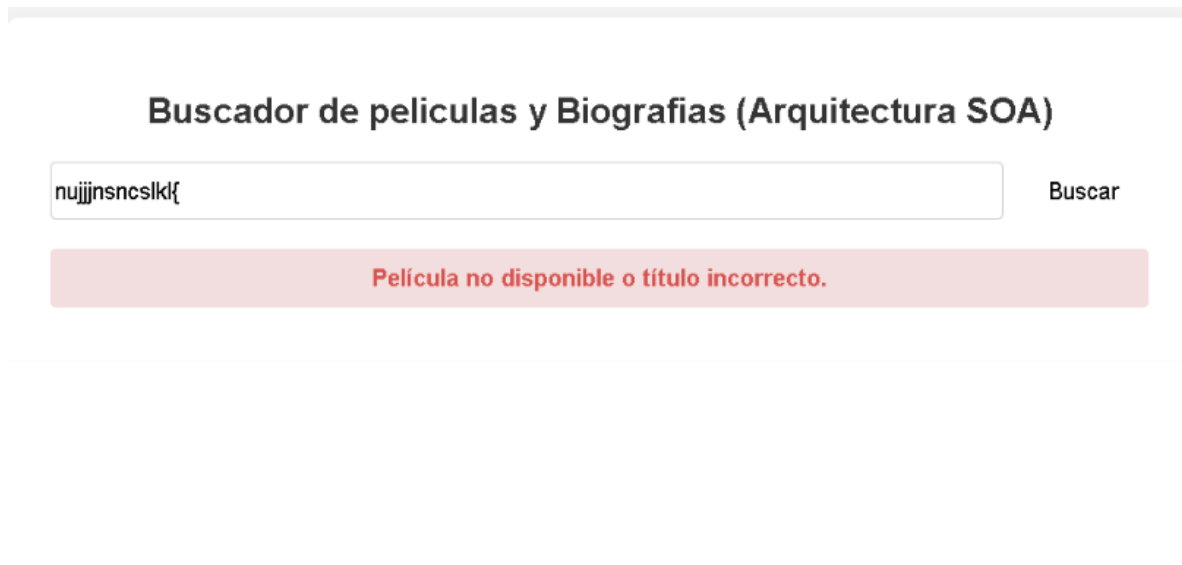
Resolución de Errores e Integración de Servicios

Durante la integración de la capa de procesos Orquestador con la capa de acceso Interfaz de Usuario y los servicios externos (OMDb y Wikipedia), se presentaron diversos incidentes que impedían el correcto flujo de la aplicación. A continuación, se documentan los errores encontrados y las soluciones arquitectónicas implementadas para garantizar la resiliencia del sistema.



Error encontrado: Al ingresar el nombre de una película inexistente, la interfaz mostraba mensajes de error que incluían etiquetas internas del código como [omdbCliente] No se encontró esta película con este título.

Solución Desacoplamiento: En una arquitectura SOA, la vista no debe enterarse de qué cliente falló ni mostrar lenguaje de desarrollo. Se modificaron los bloques catch en Servicio/omdbCliente.js y Servicio/wikipediaCliente.js para que el throw new Error() devuelva únicamente el error.message limpio. La interfaz ahora muestra mensajes amigables y comprensibles para el usuario final





The Matrix (1999)

Elenco Principal: Keanu Reeves, Laurence Fishburne, Carrie-Anne Moss

Biografía del Actor Principal(Keanu Reeves):

Biografía no disponible por el momento.

Caída en cascada por fallos en el servicio secundario (Wikipedia)

Error encontrado: Al buscar ciertas películas (ej. *The Matrix* o *Batman*), si la API de Wikipedia no encontraba una coincidencia exacta con el nombre del actor o la respuesta llegaba con un formato inesperado, la promesa fallaba y **rompía toda la ejecución**. Como resultado, ni siquiera se mostraban el póster ni los datos de OMDb (la pantalla quedaba en blanco).

Solución (Resiliencia): Se aplicó el principio de tolerancia a fallos en Proceso/orquestador.js. Se encapsuló la llamada a Wikipedia en un bloque try...catch independiente dentro del orquestador. Ahora, si Wikipedia falla o no encuentra la biografía, el orquestador captura el error localmente, asigna el texto *"Biografía no disponible por el momento"* y permite que la aplicación continúe, asegurando que los datos principales (película y póster) siempre se muestren.

Ajuste en la API: Se agregaron los parámetros &redirects=1 (para resolver variaciones en los nombres de los actores automáticamente) y &plaintext=1 (para recibir texto plano sin etiquetas HTML) en el endpoint de Wikipedia.

Error de compilación por falta de consistencia en los módulos (SyntaxError)

Error encontrado: La consola del navegador arrojó un error crítico: Uncaught SyntaxError: The requested module '../Servicio/omdbCliente.js' does not provide an export named 'obtenerPelicula'. Esto deshabilitó por completo el botón de búsqueda.

Solución: Se identificó un desajuste en el contrato entre capas. El Orquestador estaba intentando importar una función llamada obtenerPelicula, pero el módulo de servicio la estaba exportando como buscarPelicula. Se unificó la nomenclatura en los import del archivo orquestador.js para respetar el nombre real exportado, restableciendo la comunicación entre la Capa de Proceso y la Capa de Servicio.

 Practica_AWOS Public

 Pin



 main  1 Branch  0 Tags



Add file

 Code

 dan502ds9-eng fix: Manejo de errores amigables en UI, resiliencia en orquestador y ... 728d69a · 2 minutes ago 3 Commits

 Acceso	fix: Manejo de errores amigables en UI, resiliencia en orques...	2 minutes ago
 Modelo	feat: Implementación de capas Modelo, Servicios y Proceso...	3 days ago
 Proceso	fix: Manejo de errores amigables en UI, resiliencia en orques...	2 minutes ago
 Servicio	fix: Manejo de errores amigables en UI, resiliencia en orques...	2 minutes ago

ETAPA 5

Prueba de Sustitución de Servicios; Desacoplamiento.

Objetivo de la prueba: Validar el principio de bajo acoplamiento inherente a la Arquitectura Orientada a Servicios (SOA), demostrando que es posible reemplazar un proveedor de datos externo sin alterar el funcionamiento del sistema central ni la interfaz de usuario.

Procedimiento realizado: Se llevó a cabo la sustitución del proveedor de biografías, migrando de la API de *Wikipedia* a la API pública de DuckDuckGo.

Análisis Arquitectónico y Resultados:

Aislamiento del cambio: La actualización del endpoint, los parámetros de consulta y el parseo del nuevo formato JSON (extrayendo la propiedad Abstract de DuckDuckGo en lugar del extract de Wikipedia) se contuvieron de forma exclusiva en la capa de servicios, modificando únicamente un (1) archivo (Servicio/wikipediaCliente.js).

Capas intactas: Durante la sustitución, la Capa de Acceso (app.js), la Capa de Proceso (orquestador.js) y el Modelo de Datos (peliculaModelo.js) no sufrieron absolutamente **ninguna modificación**.

Transparencia del servicio: El Orquestador continuó recibiendo el objeto esperado ({ biografia: "..."}) sin importar que el motor de búsqueda subyacente fuera completamente distinto.

```

Service > JS wikipediaCliente.js > buscarBiografiaActor
You, 16 minutes ago | author (You)
1 export async function buscarBiografiaActor(actor) {
2   try {
3     if (!actor) {
4       return { biografia: "Biografía no disponible." };
5     }
6
7     // ¡Sustitución! Ahora consultamos a DuckDuckGo en lugar de Wikipedia
8     const url = `https://api.duckduckgo.com/?q=${encodeURIComponent(actor)}&format=json&no_html=1`;
9
10    const respuesta = await fetch(url);
11
12    if (!respuesta.ok) {
13      return { biografia: "Servicio de proveedor alternativo no disponible." };
14    }
15
16    const datos = await respuesta.json();
17
18    // DuckDuckGo devuelve el resumen en una propiedad llamada "Abstract"
19    if (!datos.Abstract) {
20      return { biografia: "Biografía no encontrada en el nuevo proveedor." };
21    }
22
23    // Devolvemos la variable "biografia" igual que antes, para que el Orquestador no note el cambio
24    return {
25      biografia: datos.Abstract
26    };
27
28  } catch (error) {
29    return { biografia: "Biografía no disponible por el momento." };
30  }
31 }
You, 2 days ago • feat: Implementación de capas Modelo, Servicios...
```

Evidencia de Funcionamiento:

A continuación se muestra la aplicación ejecutándose de manera estable tras la sustitución. La interfaz continúa mostrando el póster y los datos de la película provenientes de OMDb, mientras que la biografía ahora es inyectada exitosamente por el nuevo proveedor (DuckDuckGo), sin que el usuario final perciba el cambio de infraestructura.

101

Buscar



101 Dalmatians (1996)

Elenco Principal: Glenn Close, Jeff Daniels, Joely Richardson

Biografía del Actor Principal(Glenn Close):

Glenda Veronica Close, más conocida como Glenn Close, es una actriz, productora e

```
PS C:\Users\DINOBABY\OneDrive\Escritorio\Practica_AW05> git add .
>> git commit -m "refactor: Sustitución de proveedor de Wikipedia a DuckDuckGo comprobando desacoplamiento SOA"
●>> git push origin main
[main 07f1be0] refactor: Sustitución de proveedor de Wikipedia a DuckDuckGo comprobando desacoplamiento SOA
1 file changed, 9 insertions(+), 14 deletions(-)
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 753 bytes | 753.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/dan502ds9-eng/Practica_AW05.git
728d69a..07f1be0 main -> main
PS C:\Users\DINOBABY\OneDrive\Escritorio\Practica_AW05> c
```

En la captura anterior se documenta el registro en el control de versiones (Git) de la sustitución del proveedor de datos, pasando de la API de Wikipedia a la de DuckDuckGo. Como se puede observar claramente en la salida de la consola (1 file changed, 9 insertions(+), 14 deletions(-)), lograr esta migración completa de servicio requirió la modificación de **un único archivo** correspondiente a la capa de Servicios. Esto comprueba físicamente el principio de bajo acoplamiento de la arquitectura SOA, ya que el resto de las capas del sistema (interfaz gráfica y orquestador) permanecieron intactas. Los cambios fueron sincronizados exitosamente con el repositorio remoto (main -> main) para su evaluación.

Plenaria de Cierre

1. ¿Cuántos archivos tuvo que modificar para sustituir al proveedor y por qué exactamente esos?

Para sustituir a Wikipedia por DuckDuckGo, **solo se modificó un archivo** (Servicio/wikipediaCliente.js). Esto demuestra el principio de bajo acoplamiento de la arquitectura SOA: las capas de Proceso (Orquestador) y Acceso (Interfaz) ignoran quién es el proveedor externo. Toda la lógica de conexión y parseo ocurre exclusivamente dentro de la capa de Servicio.

2. ¿Qué decisión tomada en la etapa 2 facilitó o dificultó la sustitución?

La decisión que facilitó todo fue crear un **Modelo de datos interno** (peliculaModelo.js). Al hacer que el cliente traduzca los campos originales del proveedor a nuestras propias variables, logramos que la aplicación no dependa de la estructura del proveedor externo. Como el Orquestador siempre espera el mismo objeto, la transición fue transparente.

3. ¿Qué ocurriría con su aplicación si el segundo servicio tardara diez segundos en responder?

Gracias a que implementamos llamadas asíncronas (`async/await`) y un indicador visual (`cargando.style.display = "block"`), la interfaz no se congelaría; simplemente mostraría el estado de carga. Si la petición falla por tiempo de espera, el Orquestador capturaría el error, mostraría los datos de la película (póster, título) y pondría el texto alternativo en la biografía sin colapsar.

4. ¿Qué ventaja concreta de SOA pudo comprobar en su proyecto y cuál solo conoce de forma teórica?

Ventaja comprobada: El desacoplamiento. Comprobamos en la práctica que podíamos cambiar completamente de API (DuckDuckGo) sin tener que tocar ni una sola línea de la interfaz HTML/CSS ni del Orquestador. Ventaja teórica: La escalabilidad. Sabemos en teoría que cada servicio de esta arquitectura podría alojarse en servidores distintos para soportar millones de peticiones, aunque en nuestra práctica todo corrió en local.